

# Vishal Pandey | Applied ML Research

[Home](#) [About Me](#) [Blog](#)

## Prefix Tuning

18 Aug, 2025

This is the paper introduced by Xiang Lisa Li & Percy Liang from Stanford University.

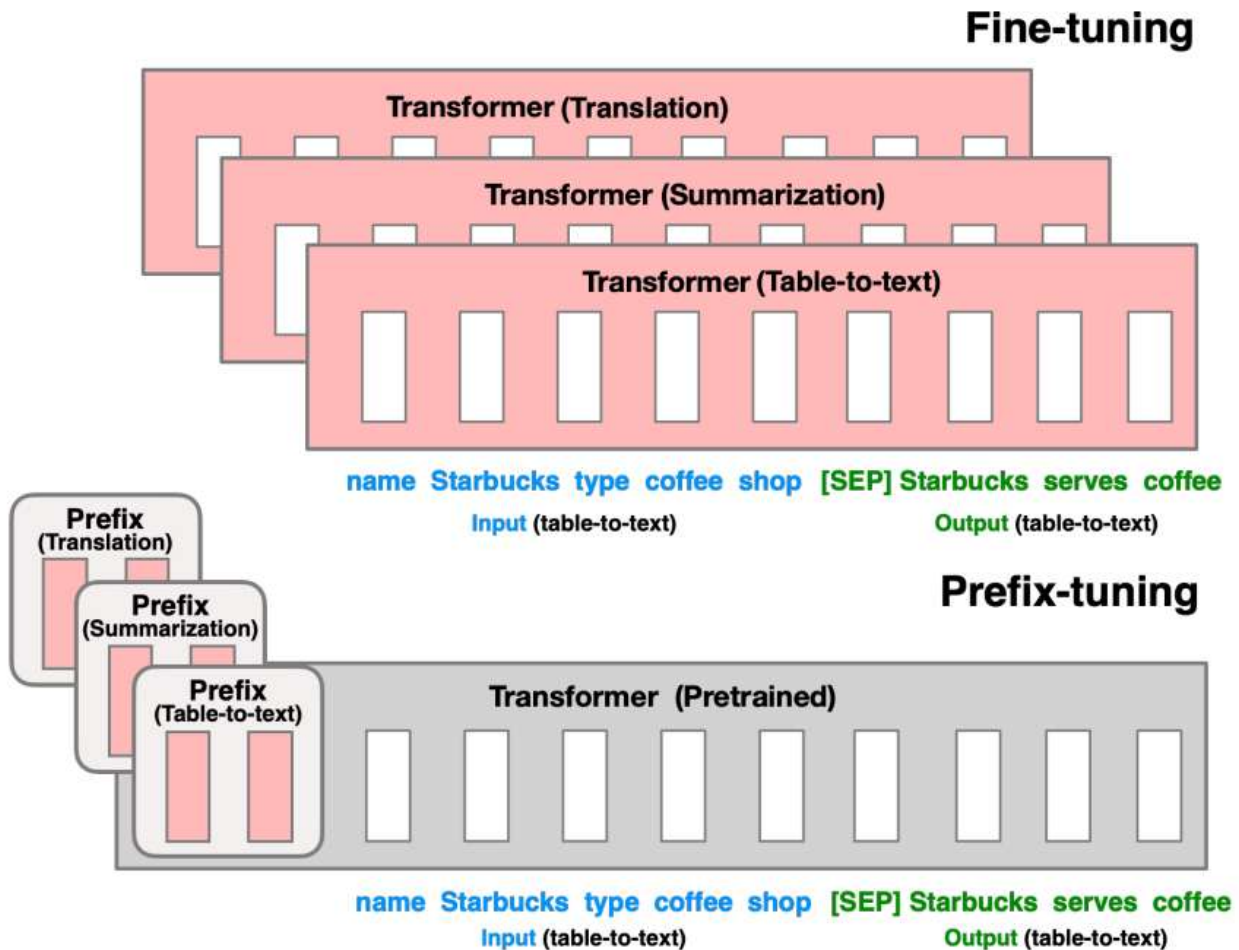
Paper Link: [Prefix Tuning: Optimizing Continuous Prompts for Generation](#)

Let's break it down.

---

### Points from Abstract:

- Fine-tuning is the one and only legitimate way to leverage large pretrained models to perform downstream tasks.
- In this paper, the authors proposed a prefix-tuning, a light-weight alternative to fine-tuning for Natural Language tasks, which keeps language models' parameters frozen, but optimizes a small continuous task-specific vector (called the prefix)
- This allows subsequent tokens to attend to this prefix as if it were virtual tokens.
- By learning only 0.1% of the parameters, the prefix tuning obtains comparable performance in the full data setting, outperforms fine-tuning in low data settings, and extrapolates better to examples with topics unseen during training.
- So, there will be a pretrained model which will be frozen, however, we will use the continuous vectors called prefixes, which are different for each task.



## 1. What problem are we solving?

You've got a big, pretrained language model  $f_\theta$ . You want it to perform a new task (summarize, translate, classify...) **without updating all of  $\theta$**  (which is expensive, slow, and can cause forgetting).

**Prefix-Tuning** says:

- Keep  $\theta$  frozen.
- Learn a tiny set of task-specific parameters  $\phi$ , called a **prefix**, that "steer" the model.

Think of the prefix as a tiny "task memory" that the model reads before it reads the actual input.

## 2. First principles: Conditioning a language model

A decoder-only LM predicts tokens by:

$$p_\theta(y_1, \dots, y_T | x) = \prod_{t=1}^T p_\theta(y_t | x, y_{<t})$$

**Full fine-tuning:** Change  $\theta$

**Prefix-Tuning:** Keep  $\theta$  fixed, and introduce auxiliary variables  $\phi$  that modify computation:

$$p_{\theta, \phi}(y_t | x, y_{< t}) = p_{\theta}(y_t | x, y_{< t}; \text{prefix}(\phi))$$

We optimize  $\phi$  to maximize likelihood on the task;  $\theta$  stays frozen.

---

### 3. Where does the prefix live? (Inside attention)

Recall a single self-attention head:

$$\text{Attn}(Q, K, V) = \text{softmax}\left(\frac{QK^{\top}}{\sqrt{d_k}}\right)V$$

Normally,  $K, V$  come from the past tokens (causal mask).

**Prefix-Tuning** augments every layer's attention with  $m$  extra key-value pairs that don't correspond to actual tokens.

For layer  $\ell$ :

$$K_{\ell}^{\star} = \begin{bmatrix} P_{\ell}^K \\ K_{\ell} \end{bmatrix}, \quad V_{\ell}^{\star} = \begin{bmatrix} P_{\ell}^V \\ V_{\ell} \end{bmatrix}$$

Where:

$$P_{\ell}^K, P_{\ell}^V \in \mathbb{R}^{m \times d_k}$$

are **learned** and task-specific (parameters  $\phi$ ).

The queries  $Q_{\ell}$  come from your actual input as usual.

So attention becomes:

$$\text{Attn}(Q_{\ell}, K_{\ell}^{\star}, V_{\ell}^{\star}) = \text{softmax}\left(\frac{Q_{\ell} \begin{bmatrix} P_{\ell}^K \\ K_{\ell} \end{bmatrix}^{\top}}{\sqrt{d_k}}\right) \begin{bmatrix} P_{\ell}^V \\ V_{\ell} \end{bmatrix}$$

**Interpretation:**

Before looking at the real context, each query attends to learned “memory slots” that encode task instructions or biases.

---

### 4. How are prefixes parameterized?

Naively, you could learn  $P_\ell^K, P_\ell^V$  directly.

Instead, the paper proposes a **reparameterization**:

Start from  $m$  virtual token embeddings:

$$E \in \mathbb{R}^{m \times d_{\text{model}}}$$

Pass them through a small 2-layer MLP to generate per-layer key-values:

$$[P_\ell^K, P_\ell^V] = \text{MLP}_\ell(E)$$

This keeps parameter count **small and stable**.

*In encoder-decoder models (like T5), you add prefixes to:*

- encoder self-attention
- decoder self-attention
- encoder-decoder cross-attention  
(with **separate**  $P$ 's for each)

## 5. Training objective (simple)

Freeze  $\theta$ . Optimize  $\phi$  using **maximum likelihood**:

$$\max_{\phi} \sum_{(x,y) \in D} \sum_{t=1}^T \log p_{\theta}(y_t \mid x, y_{<t}; \text{prefix}_{\phi})$$

Backpropagation flows only into  $\phi$ .

→ That's why Prefix-Tuning is **parameter-efficient** and **fast to adapt**.

## 6. Why does this work? (The math intuition)

Look at the attention logits with concatenated keys:

$$\frac{Q_{\ell} K_{\ell}^{\top}}{\sqrt{d_k}} \Rightarrow \frac{Q_{\ell} \begin{bmatrix} P_{\ell}^K \\ K_{\ell} \end{bmatrix}^{\top}}{\sqrt{d_k}} = \begin{bmatrix} \frac{Q_{\ell} (P_{\ell}^K)^{\top}}{\sqrt{d_k}} \\ \frac{Q_{\ell} K_{\ell}^{\top}}{\sqrt{d_k}} \end{bmatrix}$$

The first block:

$$\frac{Q_{\ell} (P_{\ell}^K)^{\top}}{\sqrt{d_k}}$$

adds **learned, content-dependent biases** to the attention distribution.

After softmax, the model can shift some probability mass toward prefix slots and mix in their values  $P_\ell^V$ .

**Result:** You can shape what each layer “pays attention to,” effectively **steering the network’s computation path without editing its weights**.

**Alternate view:** a learned basis

Each layer’s output is an affine combination of values  $V$ .

By augmenting  $V$  with learned  $P^V$ , you **expand the span** of representable outputs, like adding a **small, learnable basis** the model can project onto for this task.

## 7. How is this different from other PEFT methods?

**PEFT:** Parameter-Efficient Fine-tuning

**Prompt Tuning** (soft prompts only at input):

- Adds learnable embeddings only at layer 0 (input).
- Prefix-Tuning adds per-layer key/value pairs, giving deeper control.
- Empirically stronger on generation with fewer params.

**Adapters:**

- Insert small MLP blocks inside each layer and train them.
- Strong, but requires forward changes and slightly more latency.

**LoRA:**

- Learn low-rank updates to weight matrices:  $W_q, W_k, W_v, W_o$
- Fine-grained and strong on many tasks; touches more places than prefixes.

**Tradeoff:**

Prefixes = no weight injections, just extra KV, simple to serve and compose.

## 8. Quick Recap (Mental Model)

- A Transformer layer decides via **attention** *where to look*.
- **Prefix-Tuning** gives it a **learned place to look**, a tiny, per-layer **memory** that encodes *what this task wants*.
- Training only shapes that memory, the big model (parameters  $\theta$ ) remains **frozen**.

## 9. Pytorch-ish Pseudocode

```
# given frozen model, Learnable PrefixModule
Kp_l, Vp_l = prefix_module() # list over layers, shapes: [L, m, d_k]
for l in layers:
    K_star = cat([Kp_l[l], K_l], dim=1)
    V_star = cat([Vp_l[l], V_l], dim=1)
    out = attn(Q_l, K_star, V_star) # rest unchanged
loss = cross_entropy(outputs, targets) # backprop ONLY into prefix_module
```

## 10. Brain-Teaser

**If we only added learnable keys  $P^K$ , but not values  $P^V$ , what behavior would you expect during attention, and why?**

Queries (  $Q$  ) can still produce attention logits with those prefix keys:  $\text{logits} \propto Q(P^K)^\top$ . So, the attention distribution can still shift probability mass toward the prefix slots.

**But** since the corresponding values  $P^V$  are missing (or zero), attending to them doesn't inject any new information. It's like pointing to an **empty memory slot**.

That means attention is **wasted** on meaningless content, and the model can't effectively adapt to the task.

The real issue isn't "*no bias*", it's "*no useful content*."

Attention requires both:

- **Where to look** (keys (  $K$  ))
- **What to retrieve** (values (  $V$  )) Without values, the prefix can't **steer computation**.

The refinement: **keys alone create positions, but without values, those positions hold nothing, so the model can't change behavior.**

### A. Normal Attention (No Prefix)

The attention mechanism:

$$\text{Attn}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

With:

- $Q \in \mathbb{R}^{T \times d_k}$
- $K \in \mathbb{R}^{T \times d_k}$
- $V \in \mathbb{R}^{T \times d_v}$

Each row of  $\frac{QK^T}{\sqrt{d_k}}$  gives attention logits  $\rightarrow$  softmax  $\rightarrow$  weights to average rows of  $V$ .

So:

- **Keys = where to look**
- **Values = what to retrieve**

## B. With Full Prefix ( $P^K, P^V$ )

Augment keys and values:

$$K^* = \begin{bmatrix} P^K \\ K \end{bmatrix}, \quad V^* = \begin{bmatrix} P^V \\ V \end{bmatrix}$$

Now attention becomes:

$$\text{Attn}(Q, K^*, V^*) = \text{softmax}\left(\frac{Q \begin{bmatrix} P^K \\ K \end{bmatrix}^T}{\sqrt{d_k}}\right) \begin{bmatrix} P^V \\ V \end{bmatrix}$$

Queries can now spread attention across both:

- $P^K$ : task-specific prefix keys
- $K$ : regular keys from input

If attention shifts to  $P^K$ , the model retrieves corresponding  $P^V$ , which is **learnable**  $\rightarrow$  **task-specific information enters computation**.

## C. With Only Prefix Keys $P^K$ (No Prefix Values)

We still augment keys:

$$K^* = \begin{bmatrix} P^K \\ K \end{bmatrix}$$

But now the values are:

$$V^* = \begin{bmatrix} 0 \\ V \end{bmatrix}$$

So attention becomes:

$$\text{Attn}(Q, K^*, V^*) = \text{softmax}\left(\frac{Q \begin{bmatrix} P^K \\ K \end{bmatrix}^\top}{\sqrt{d_k}}\right) \begin{bmatrix} 0 \\ V \end{bmatrix}$$

## D. Why This Collapses

The logits term:

$$\frac{Q \begin{bmatrix} P^K \\ K \end{bmatrix}^\top}{\sqrt{d_k}} = \begin{bmatrix} \frac{Q(P^K)^\top}{\sqrt{d_k}} \\ \frac{QK^\top}{\sqrt{d_k}} \end{bmatrix}$$

So:

- Probability mass **can shift** toward prefix slots
- But their **values are all zero**, so:

$$\text{Attn}(Q, K^*, V^*) = \underbrace{\alpha_{\text{prefix}}}_{\text{wasted}} \cdot 0 + \alpha_{\text{real}} \cdot V = \alpha_{\text{real}} \cdot V$$

Net effect:

- Attention on the prefix is **wasted**
- Model only gets info from regular  $V$
- So it can't adapt to the new task

## What happens during training?

The model learns to avoid prefix keys entirely  $\rightarrow$  logits pushed to  $-\infty$ , prefix gets ignored.

## Intuition Check:

Keys without values create **empty memory addresses**. Attention can point to them, but there's nothing to retrieve.

---

## Soft Prompt Implementation

```
import torch
import torch.nn as nn

class SoftPrompt(nn.Module):
    def __init__(self, prompt_length, embedding_dim):
        super(SoftPrompt, self).__init__()
        self.prompt_embeddings = nn.Embeddings(
```



```
        prompt_length, embedding_dim
    )

    def forward(self, input_ids):
        prompt_ids = torch.arange(
            self.prompt_embeddings.num_embeddings,
            device = input_ids.device
        )
        prompt_embeddings = self.prompt_embeddings(prompt_ids)
        return torch.cat(
            (prompt_embeddings, input_ids), dim = 1
        )
```

---

## Prefix Tuning Implementation

```
import torch
import torch.nn as nn

class PrefixTuning(nn.Module):
    def __init__(self, prefix_length, embedding_dim, num_layers):
        super(PrefixTuning, self).__init__()
        self.prefix_embeddings = nn.Embeddings(
            prefix_length * num_layers, embedding_dim
        )
        self.prefix_length = prefix_length
        self.num_layers = num_layers

    def forward(self, input_ids):
        batch_size = input_ids.size(0)
        total_prefix_length = self.prefix_length * self.num_layers
```



Powered by Bear 